Google Cloud H2S

AI Builder Cup

Build in JAPAC.
Build for the world.



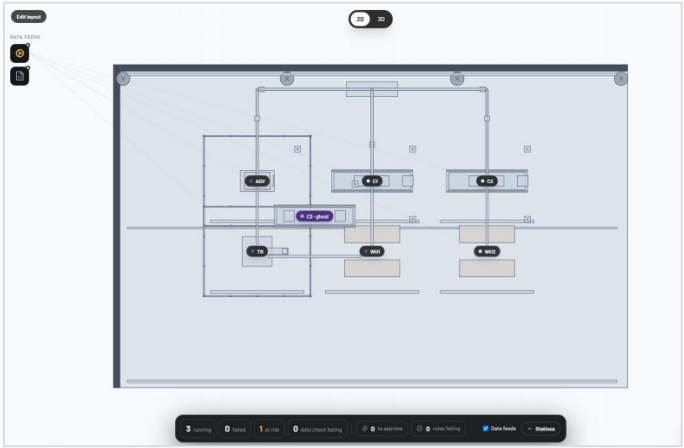
Team Details

a. Team name: Tolerance
b. Team leader name: Shreyash Sharma
c. Problem Statement: Manufacturing. Plants run on scattered knowledge: manuals, orders, machine logs. When a station fails, nobody can quickly tell why, or whether a fix is safe. We built an agentic platform that learns a plant from its own documents and data, finds the cause, and acts only inside limits the owner sets.

Brief about the idea

An agentic platform for manufacturing. It learns a plant from the plant's own documents and data, finds why a station failed, and acts only inside limits the owner sets.

- **Nine agents, one gateway.** Four read the documents, connect the systems and build the model. Five run it: Diagnosis, Verifier, Planner, Warden and Sentinel.
- **Evidence before action.** Every fact keeps the exact quote it came from. Instructions hidden in a file are removed, never obeyed. The Verifier recomputes every number a diagnosis cites.
- **Autonomy is a dial.** Warden scores each action 0 to 100. The owner sets two lines: refuse, ask a person, or run alone. A hard limit from the documents is never crossed.
- **Built on Google Cloud.** Gemini on Vertex AI in every agent, Cloud Run, Firebase sign-in.



The live floor: plan, data feeds and status.

9	465	10
agents	incidents backtested	live system types

Opportunities

How is it different from other ideas?

- It learns from the plant's own documents and data, not from a fixed template.
- It finds the systems your documents name (MES, ERP, MongoDB, MQTT, MCP servers) and asks for each one by name.
- Autonomy is a setting the owner controls, enforced by policy, not by a prompt.

How will it solve the problem?

- It compiles scattered manuals, logs and orders into one checked model. Every fact has its quote.
- When a station fails, Diagnosis, Verifier, Planner and Warden answer in about 20 seconds.
- Sentinel watches data, services and certificates. Playbooks and channels reach the right person.

USP of the solution

- Evidence before action: a wrong number is sent back, a broken hard limit blocks the action.
- Two lines you set, tried first on 465 recorded incidents before you trust them.
- Every step is recorded. Gemini works in every agent. Plain rules decide every safety result.

Plants already have the data and the experts. Tolerance connects them, checks its own answers, and leaves the authority with the people.

List of features offered by the solution

Guide onboarding

Asks the plant name, reads the documents, settles conflicts, draws the plant.

Document Reader

Facts with exact quotes. Hidden instructions are removed first.

Live connectors

PostgreSQL, MySQL, SQL Server, Oracle, MongoDB, MQTT, Kafka, OPC UA, REST. Test, then connect.

Your own MCP servers

They join the same hub. Only read-only tools can run.

Floor in 2D and 3D

A Pascal plan and 3D view, with every data feed joined to its stations.

Layout editor

Move, add and delete stations. Draw how parts flow.

Operations

Diagnosis, Verifier, Planner and Warden in one run, step by step.

Warden gate

Score 0 to 100, your two lines, hard limits, a backtest.

Sentinel

Plain rules on data, services and certificates. Maintenance windows.

Playbooks

Draw what should happen when a station fails.

Approval channels

Slack, Teams, Discord, Telegram or a webhook.

Private by design

Firebase sign-in, a private plant per user, encrypted passwords, usage caps.

Process flow diagram or Use-case diagram

Set up once

Give the plant name and documents → Connect the systems the documents name → The model is compiled, each fact with its quote → The owner confirms what was understood → The floor goes live in 2D and 3D → The owner sets the two lines

When a station fails

Diagnosis
finds the cause with read-only tools

→ **Verifier**
recomputes every number it cites

→ **Planner**
proposes the fix with exact settings

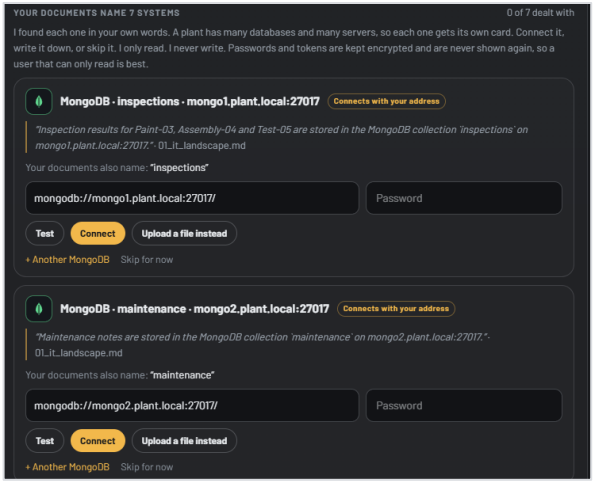
→ **Warden**
hard limits, then a score from 0 to 100

→

- **Blocked or refused:** does not run
- **A person approves:** in the app or chat
- **Runs by itself:** inside your limit

Then: the Action Broker applies only what the gate allows, in a simulator. Every model call, tool call and decision is recorded. **Who uses it:** the plant owner sets it up and the limits, an operator reads the floor, a person approves the middle zone.

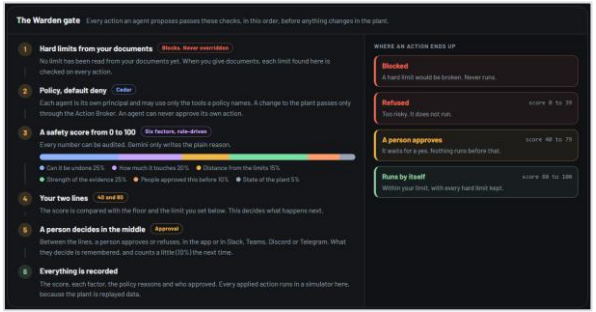
Wireframes/Mock diagrams of the proposed solution (optional)



The Guide asks for each system the documents name, with the sentence that names it.



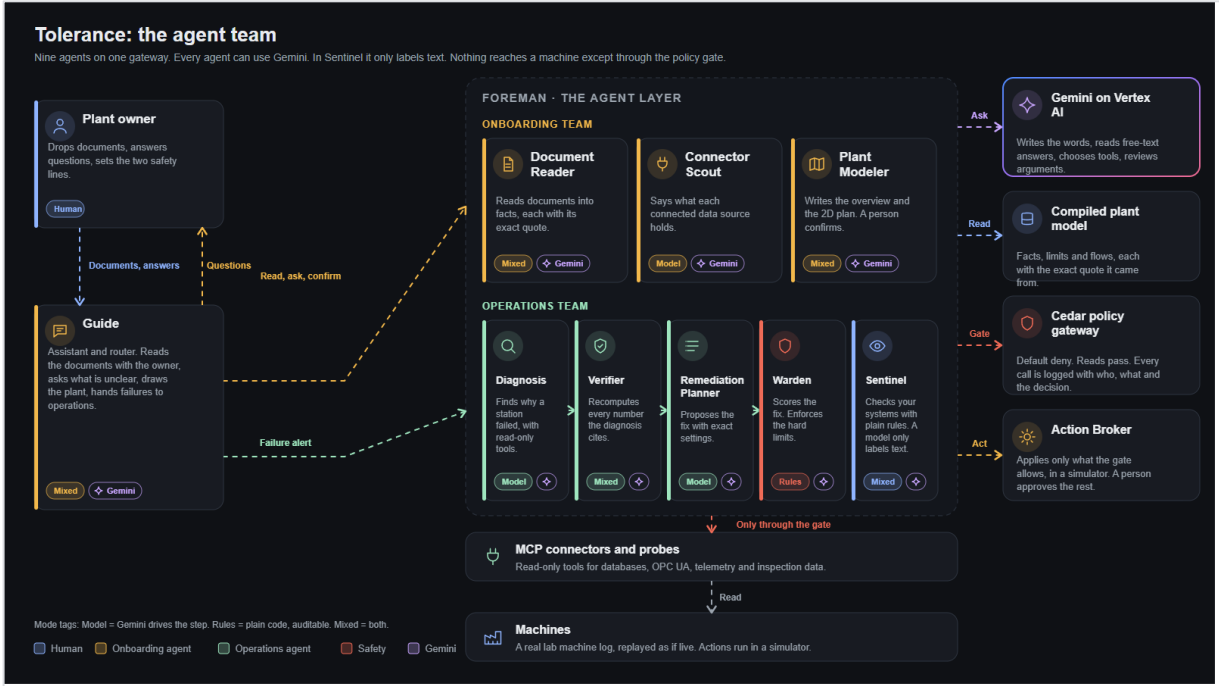
The layout editor: move and add stations, draw part flows.



The Warden gate: every rule an action must meet.

These are screens of the running prototype, not sketches.

Architecture diagram of the proposed solution



The agent team: owner, Guide, onboarding and operations agents, shared services, policy gateway and machines.

How it fits together

- **One container on Cloud Run:** the web app and the API.
- **Foreman:** runs the nine agents, with a contract card and a record of every step.
- **Gemini on Vertex AI** writes the words, reads answers and picks tools.
- **Cedar policy gateway:** default deny; nothing reaches a machine except through it.
- **MCP hub:** built-in and the owner's own servers; only read-only tools run.
- **Data lake:** DuckDB, loaded once and kept current by connectors.

Technologies to be used in the solution

Google Cloud

- Gemini on Vertex AI (Gemini 2.5 Flash and the flash-lite family)
- Cloud Run, one always-on instance
- Firebase Authentication (Google, email, guest)
- Cloud Build and Artifact Registry

Agents and safety

- Foreman agent layer, Gemini function calling
- Cedar policy, default deny
- Model Context Protocol (hub and client)
- Rule-driven Warden and Sentinel

Data and connectors

- DuckDB lake, dlt loaders
- SQLAlchemy: PostgreSQL, MySQL, SQL Server, Oracle
- pymongo, MQTT, Kafka, OPC UA (asyncua)
- Encrypted vault for passwords

Web

- Next.js 16 and React 19
- Pascal Editor viewer (2D plan, 3D)
- three.js, React Flow
- FastAPI backend

Gemini is called through one gateway: a daily cap, model rotation when a model is busy, recorded answers as a fallback, and a visible notice when Gemini is under load.

Open source is used under its own licence; the licences are listed in the repository.

Estimated implementation cost (optional)

Cloud Run	One always-on instance (2 vCPU, 2 GiB) serves the web app and the API.
Gemini on Vertex AI	A daily cap of 1,500 calls and a quota per visitor. Recorded answers replay without a call.
Firebase Authentication	Free tier covers sign-in with Google, email and guest.
Cloud Build, Artifact Registry	A build per deploy.
Data	Files and DuckDB inside the container. No managed database yet.

Kept small on purpose

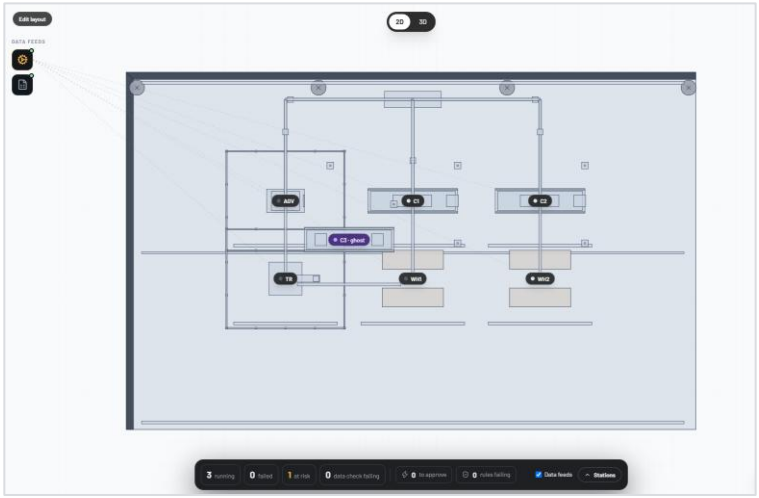
The build is planned to stay under $100 of the $300 credit.

Caps that keep it there

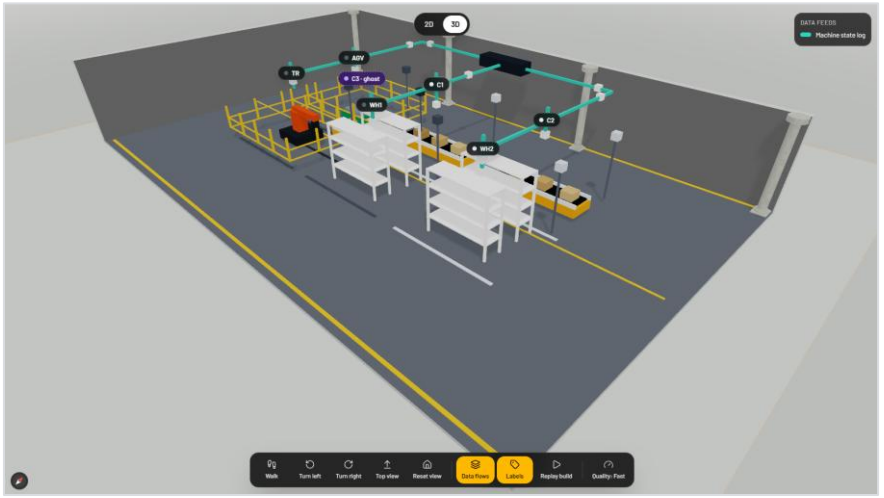
- A daily limit on Gemini calls
- A quota per visitor, tighter for guests
- Recorded answers replay for free

To grow: one lake per plant in Cloud SQL or Cloud Storage, more instances under load, and a model budget per customer.

Snapshots of the prototype



The demo plant in 2D: data feeds, name tags and the status panel



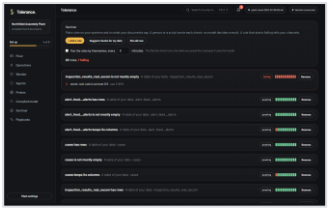
The same plant in 3D, with its data flows



The layout editor



The Warden gate



Sentinel rules

Prototype Performance report/Benchmarking

465
real fault-injection incidents replayed through Warden

75%
of those diagnoses were right

~18 s
failed station to a scored plan, measured locally

0.2 s
a read-only MCP tool call, measured locally

Warden with the default lines (40 and 80)

149 would run alone: 120 right, 29 wrong

316 would wait for a person

The incident set is software faults (RCAEval). There is no history from a real plant yet.

Checks that pass today

- 12 automated suites: policy, agents, Sentinel, probes, connectors, vault, MCP.
- Real PostgreSQL and MQTT connected, loading only what is new.
- A remote MCP server: a read-only tool runs, an acting tool is denied.
- Not yet tested live: Kafka, Oracle, SQL Server, MySQL.

Figures come from the running prototype on one computer. Gemini's speed varies with load, so the app shows a notice when Gemini is busy and the plain rules answer instead.

Benchmarks: where Tolerance beats the baseline, and by how much

Q1 **Does the root-cause ranking beat BARO on metric-only faults?**

RCAEval RE1: 298 cases, four faults (CPU, memory, delay, loss), three systems.

Q2 **Does a second source (traces, logs) help on code-level faults?**

RCAEval RE3: 90 cases, five fault types, three systems.

Q3 **Does the safety gate bound the harm of a wrong action?**

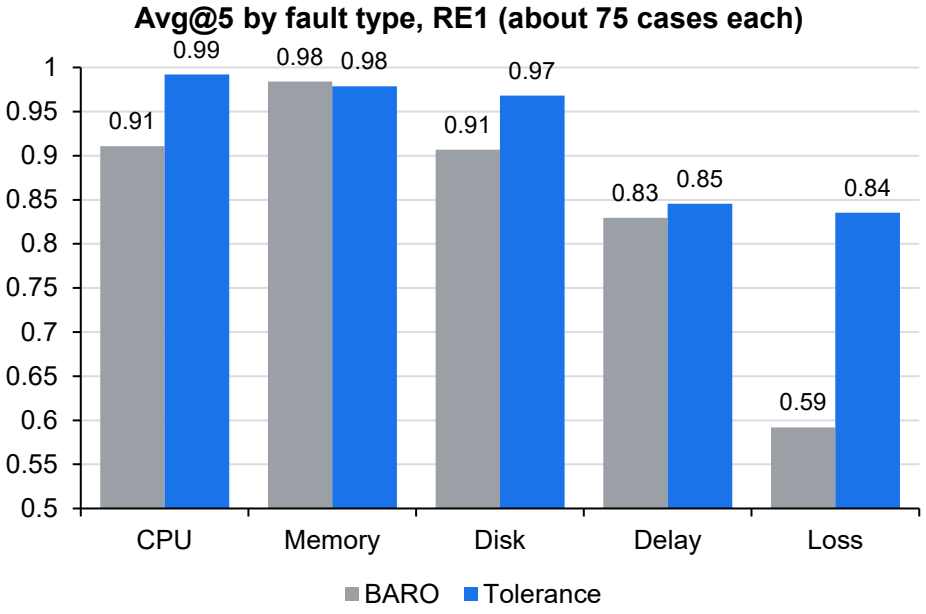
465 recorded incidents scored by Warden, with the diagnosis marked right or wrong.

Protocol

- Baseline: the authors' BARO code, unmodified, scored the way RCAEval scores it (Avg@5).
- Every interval is a 95% bootstrap over cases (10,000 resamples). Differences are paired: both methods on the same cases.
- The fusion constant was fixed before we looked at results. Every variant is reported, not the best one.
- No Gemini anywhere in Q1 and Q2. Raw per-case results and scripts are in the repository.

Headline: +0.19 on Train Ticket (Q1), +0.19 on Sock Shop with a second source (Q2), and zero harm from wrong actions at 32% autonomy (Q3).

Q1: a significant gain on metric-only faults, +0.19 on Train Ticket



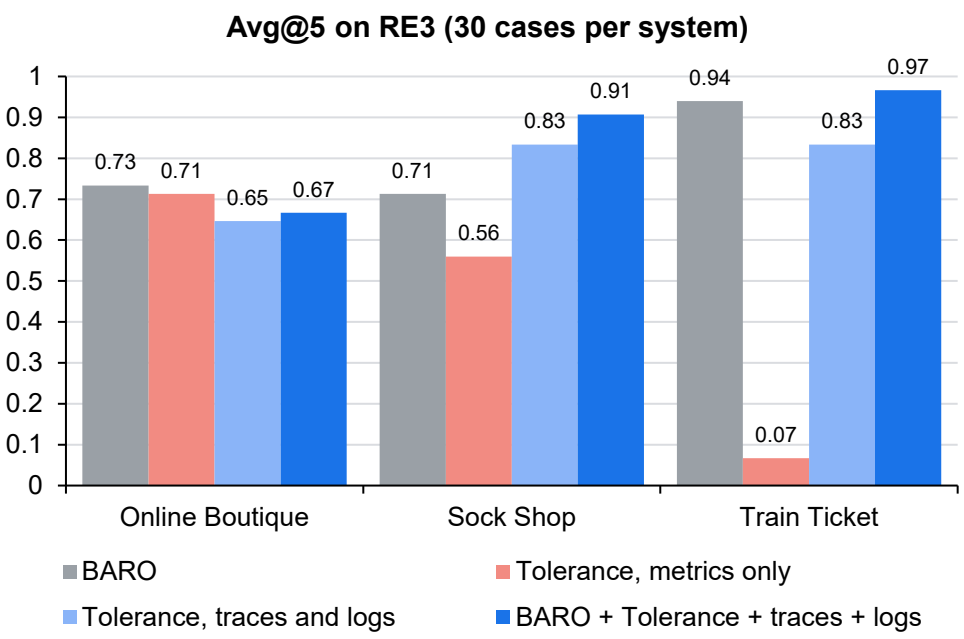
Difference from BARO, Avg@5 (95% interval)

Subset	n	Tolerance minus BARO
all	298	+0.08 [+0.05, +0.12]
Train Ticket	100	+0.19 [+0.10, +0.27]
Online Boutique	98	+0.04 [-0.01, +0.10]
Sock Shop	100	+0.02 [-0.01, +0.05]

- BARO is already at 0.96 on Sock Shop, so there is little room.
- Train Ticket gains most: 0.66 to 0.85 (BARO's paper reports 0.79).
- Packet loss gains most by fault: 0.59 to 0.84.
- Pooled rows use the four faults the BARO paper reports.

Significant on the pooled set (+0.08) and on Train Ticket (+0.19). Sock Shop is already at 0.96 for the baseline.

Q2: a second source lifts Sock Shop by +0.19 on code-level faults



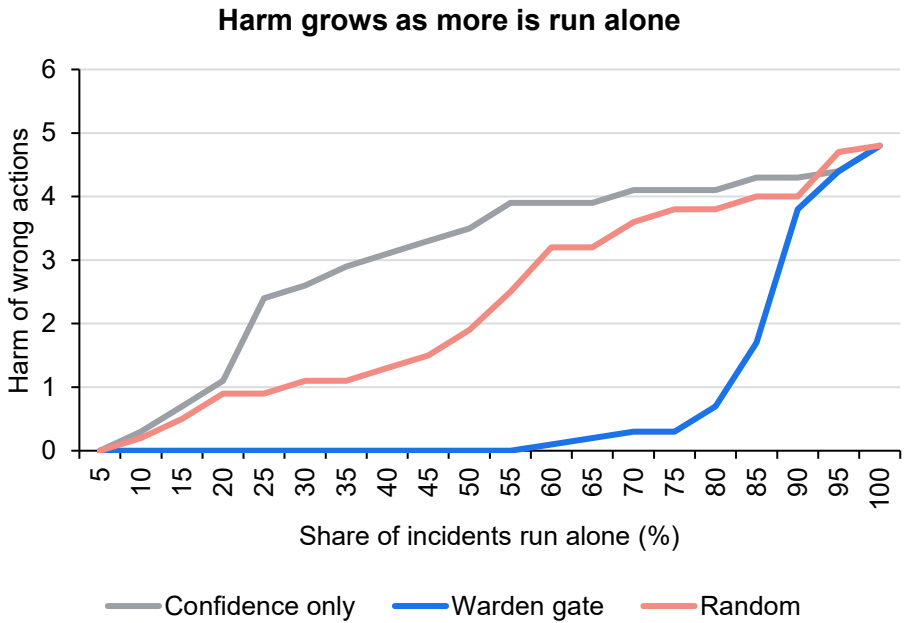
BARO + Tolerance + traces + logs, minus BARO (95% interval)

Subset	n	Difference
all	90	+0.05 [-0.00, +0.11]
Sock Shop	30	+0.19 [+0.11, +0.29]
Train Ticket	30	+0.03 [-0.03, +0.10]
Online Boutique	30	-0.07 [-0.17, +0.05]

- Code-level faults live in logs and traces, so metrics alone are not enough.
- Fusing all sources wins on Sock Shop and matches the baseline on Train Ticket.

Sock Shop +0.19 (interval +0.11 to +0.29), Train Ticket level with the baseline. Online Boutique is the system we work on next. 30 cases per system.

Q3: the gate keeps harm at zero while a third of incidents run alone



At your default lines (465 incidents)

Policy	Run alone	Wrong	Harm
Run everything alone	465	116	4.8
Random 149	149	33	1.1
Confidence only	149	19	2.6
Warden gate	149	29	0.0

- Harm weighs each wrong action by how hard it is to undo. The gate lets only easily undone actions run alone.
- Gate and diagnosis confidence together, for 100 run alone: 11 wrong and harm 0.0, against 9 wrong and harm 1.1 for confidence alone.

Confidence says whether to trust the diagnosis. The gate says whether the action is safe to run alone. Tolerance uses both.

What the results cover, and the next tests

What the results cover

- Software incidents on RCAEval. A factory pilot is the next step.
- The probes were developed on these cases, so RE2 is the held-out test.
- Harm uses the action table's reversibility values, not measured damage.
- The backtest uses the metrics-first diagnosis, not Gemini.
- Our run of BARO on Train Ticket is 0.65; the paper reports 0.79.

What we run next

- RCAEval RE2 (270 cases), the paper's main table, as a held-out test.
- The Gemini agent on RE3, where log text should matter, inside the credit budget.
- The other baselines (CIRCA, RCD, CausalRCA) once their dependencies fit.
- A customer's own incident history replayed through Warden.

Reproduce it: python -m bench.research reads the per-case rankings in bench/results and the 465 incidents in seed. No downloads, runs in seconds.

Additional Details/Future Development (if any)

Tune Gemma for Sentinel	Gemini sorts short texts into labels today. We plan to fine-tune Gemma for that one classification job, with plain rules still judging the result.
Reach private plant networks	Probes or a tunnel so a cloud copy can read systems inside a factory network.
A pilot with a real plant	Replace the replayed lab log and software incidents with a customer's own history.
Write actions behind Warden	Let remote MCP tools act, only through the gate and with a person's approval.
More live checks	Test Kafka, Oracle, SQL Server and MySQL against live servers. Add roles for approvers and many plants per owner.

Provide links to your:

- GitHub Public Repository: https://github.com/shreyash-sharma/tolerance
- Demo Video Link (3 Minutes): to be added
- Final Product Link: https://tolerance-749288317336.asia-southeast1.run.app

How to try it

1

Open the final product link.

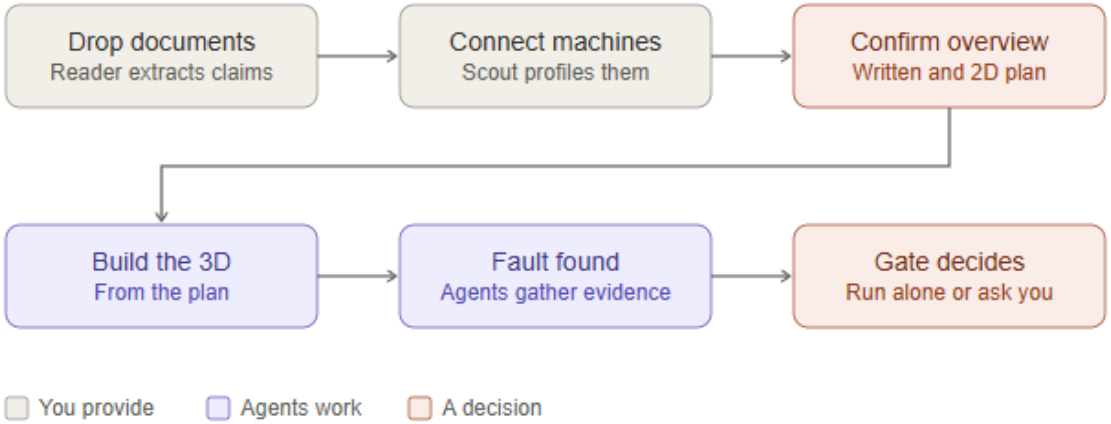
2

Choose Register, Sign in, or Try it as a guest. A guest gets a private plant with usage caps.

3

Give a plant name and drop documents, or load the made-up plant that has MES, MongoDB, ERP and IoT data.

Appendix: the journey, from documents to a decision

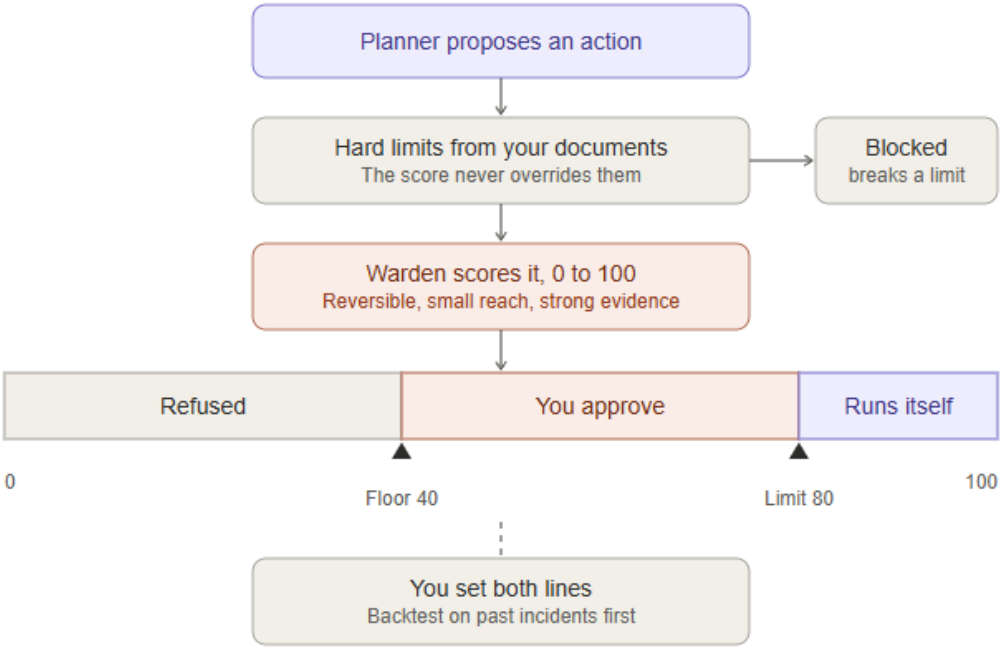


Boxes in grey are what you provide, in blue what the agents do, in red a decision.

Six steps

- You provide the documents and name the systems that hold the data.
- The agents read, connect and compile. You confirm what was understood.
- The plant is built from the checked facts, in 2D and 3D.
- When a fault is found, the agents gather evidence. The gate decides: it runs alone, or it asks you.

Appendix: the safety gate, limits first and then the score



Why this order

- A hard limit from your documents comes first. A score never overrides it.
- Warden then scores the action from 0 to 100 on six factors: can it be undone, how much it touches, distance from the limits, strength of the evidence, what people approved before, and the state of the plant.
- You set the floor and the limit. Below the floor it is refused. In the middle you approve. At or above the limit it runs by itself.
- You see what your lines would have done on 465 recorded incidents before you trust them.

Appendix: the onboarding team, four agents that learn your plant

Guide

Model and rules · Gemini

Use case: A plant's documents disagree and have gaps. Only the owner can settle them.

Solution: Runs the Document Reader, Connector Scout and Plant Modeler, then asks the owner one question at a time. Gemini writes the words and reads free-text answers. Code checks each answer before it changes anything.

Outcomes: Settled conflicts · Confirmed limits · What the plant makes · A confirmed overview · A drawn layout · A tour of the app

Must never: Apply an answer that does not match an open question; Write to the plant; Obey instructions found inside documents

Document Reader

Model and rules · Gemini

Use case: Manuals, work instructions and safety sheets hold the limits and the structure of the plant.

Solution: Gemini extracts claims; code checks every quote against the page; instructions hidden in documents are removed first.

Outcomes: Claims with quotes · Conflicts between documents · Missing facts · Hidden instructions flagged

Must never: Obey instructions found inside documents; Write to the plant

Connector Scout

Model-driven · Gemini

Use case: A plant has ERP, MES, PLC and file data. Someone must say which is which.

Solution: MCP servers list sources and tables. Gemini names the role of each table from its columns.

Outcomes: Role of each table · Which sources are live · What data is missing

Must never: Write to the plant; Read rows outside SELECT

Plant Modeler

Model and rules · Gemini

Use case: The 3D model must come from facts a person has checked.

Solution: Stations from data, material flow from the documents, limits from the documents. Gemini writes the overview and each sentence cites its source.

Outcomes: Written overview · 2D plan · Limits list · Confirmation gate

Must never: Build the 3D before a person confirms

Appendix: the operations team, four agents that keep it running

Diagnosis

Model-driven · Gemini

Use case: A station stopped. Someone must say why, fast.

Solution: Gemini picks MCP tools, reads the results and submits a cause with evidence. Cedar allows each call.

Outcomes: Cause · Evidence per claim · Tool trace

Must never: Write to the plant; Call a tool Cedar denies

Verifier

Model and rules · Gemini

Use case: A model can sound sure and be wrong.

Solution: Rule checks recompute every cited number from the tool results. Gemini then reviews the argument. A hard check cannot be talked away.

Outcomes: Supported or not · Issues list · Evidence score

Must never: Approve a claim whose numbers do not match

Remediation Planner

Model-driven · Gemini

Use case: After a cause is known, someone must choose a fix that respects the plant's limits.

Solution: Gemini proposes actions from the diagnosis and the limits. Warden scores each. Nothing is applied here.

Outcomes: Action list · Exact settings · Why this action

Must never: Write to the plant; Propose a setting above a hard limit as the first choice

Warden

Rule-driven · Gemini

Use case: Some fixes are small and some can stop a line. The user decides how far the system may go alone.

Solution: Six factors give a score from 0 to 100. The user sets a floor and a limit. Cedar enforces them at the Action Broker.

Outcomes: Score with factors · Auto, approve or refuse · Backtest on past incidents

Must never: Let a high score override a hard limit

Appendix: plain rules, your playbooks, your channels

Sentinel

Model and rules · Gemini

Use case: A plant's IT side fails quietly: a feed stops, a column changes, a service is slow. A model should not be the judge of that.

Solution: Plain rules check a field against a value on your data tables, a web address, a port, a service or a certificate. A schedule runs them, maintenance windows keep planned work quiet, and an impact map shows which stations lose data. A rule that fails or recovers tells your channels.

Outcomes: Pass or fail for each rule · The stations that lose data · A message when a rule fails or recovers

Must never: Write to the plant; Let a model say pass or fail; Reach a private network address

Playbooks

Rule-driven

Use case: Each plant reacts to a failure in its own way.

Solution: The owner draws a trigger, checks, a decision and actions on a canvas. Code walks it. Warden scores any action that changes the plant.

Outcomes: A trace of every block · Actions scored by Warden

Must never: Skip Warden for an action that changes the plant

Channels

Rule-driven

Use case: A person who is not in the app must still know when to approve.

Solution: Warden messages Slack, Teams, Discord, Telegram or a webhook. Addresses stay on the server. Private addresses are refused.

Outcomes: Approval requests · Blocked actions · Failing rules

Must never: Send to a private network address; Show an address back in the app

Layout editor

Rule-driven

Use case: The drawn floor is never exactly right.

Solution: A node canvas next to the live floor: move, add and delete stations, rename them, and draw how parts flow. Every change is saved at once and the plan and 3D view follow.

Outcomes: Moved, added or deleted stations · Part flows between stations

Must never: Change data or limits

Google Cloud H2S

AI Builder Cup

Build in JAPAC.
Build for the world.

Thank you

